JAVA DSA • INTERVIEW PREPARATION

SLIDING WINDOW

ULTIMATE INTERVIEW HANDBOOK

Concepts • Recognition • Formulas • Dry Runs • Complete Java Code • Problem Patterns

Is handbook ka goal: sirf solution yaad karna nahi. Problem statement padhkar window define karna, expand/shrink ka rule banana, data structure choose karna aur correct Java implementation likhna seekhna.

Coverage: Fixed window, variable window, at-most/exactly-K, frequency maps, sets, monotonic deque, two pointers, string windows, numeric windows, 2D extension, streaming use-cases, common mistakes, interview decision guide and practice roadmap.

Designed for beginner-to-advanced DSA practice

How to use this handbook

Is PDF ko sequence mein padho. Pehle window ka mental model samjho, phir fixed-size pattern, phir variable-size pattern. Advanced problems tabhi karo jab tum bina dekhe basic templates likh pao.

Study pass	Kya karna hai	Expected result
Pass 1 — Understand	Definitions, diagrams, formulas aur dry runs padho.	Window kya represent karti hai, samajh aa jaye.
Pass 2 — Implement	Har template ko IDE mein khud type karo; copy-paste mat karo.	Syntax aur pointer movement yaad ho.
Pass 3 — Explain	Har line ka reason bolo: add kyun, remove kyun, answer kab update?	Interview mein approach explain kar pao.
Pass 4 — Variations	Same pattern ke 3–5 problems solve karo.	Naye statement mein pattern identify kar pao.
Pass 5 — Debug	Empty input, k=0, k>n, duplicates, negative numbers test karo.	Edge cases handle kar pao.

Contents

- 1. Sliding Window ka mental model
- 2. Pattern recognition: kab use karein, kab nahi
- 3. Fixed-size window: core template and variations
- 4. Variable-size window: expand and shrink
- 5. At most K vs exactly K
- 6. HashMap, HashSet and frequency arrays
- 7. Monotonic deque: maximum/minimum in every window
- 8. String problems and character windows
- 9. Numeric, array and streaming applications
- 10. Advanced patterns and 2D extension
- 11. Brute force to optimized thinking
- 12. Java implementation toolkit and debugging
- 13. Interview problem catalogue with hints
- 14. Complexity, pitfalls and revision sheet

1. Sliding Window ka mental model

Sliding Window ek technique hai jisme array, string ya stream ke ek continuous (contiguous) segment ko maintain kiya jata hai. Segment ke left aur right boundaries ko move karke baar-baar poora segment calculate karne se bacha jata hai.

Window ka simple example

Array: [2, 1, 5, 1, 3, 2], window size k = 3. Pehli window [2,1,5] ka sum 8. Agli window [1,5,1] banate waqt purani window ka pehla element 2 remove aur naya element 1 add hota hai. Naya sum = 8 - 2 + 1 = 7. Har baar 3 elements ka sum dobara nahi nikalna.

Window	Action	Sum
[2, 1, 5]	Initial sum = 2+1+5	8
[1, 5, 1]	8 - outgoing 2 + incoming 1	7
[5, 1, 3]	7 - outgoing 1 + incoming 3	9
[1, 3, 2]	9 - outgoing 5 + incoming 2	6

Core idea: Window ke andar ka state maintain karo. Jab right move kare, incoming item ka effect add karo. Jab left move kare, outgoing item ka effect remove karo.

Contiguous ka matlab

- Subarray: elements continuous hone chahiye. Example [1,5,1] valid; [2,5,3] original order mein continuous nahi, isliye subarray nahi.
- Subsequence: beech ke elements skip ho sakte hain. Sliding Window aam taur par subsequence ke liye nahi hoti.
- Window [left, right] inclusive hai toh length = right - left + 1.

Do pointer, ek window

left window ka start hai; right window ka end hai. Window ke current elements a[left] se a[right] tak hain. Fixed-size pattern mein size constant rehta hai. Variable-size pattern mein condition ke hisaab se size badhta-ghatta hai.

Window length formula

```
int windowLength = right - left + 1;
```

2. Pattern recognition: kab Sliding Window use karein?

Statement mein ye clues dekho

Clue in question	Likely approach	Reason
Subarray / substring of size k	Fixed window	Length pehle se fixed hai.
Maximum/minimum sum of k consecutive elements	Fixed window sum	Previous sum se next sum milta hai.
Longest substring with no repeated character	Variable window + Set/Map	Invalid hone par left move hota hai.
Smallest subarray with sum at least target	Variable window	Positive numbers mein sum threshold se control hota hai.
At most K distinct values	Variable window + frequency map	Distinct count maintain hota hai.
Maximum of every window of size k	Monotonic deque	Har window ka max efficiently maintain hota hai.

Sliding Window kab suitable nahi?

- Agar question arbitrary indices ka hai, contiguous segment nahi, toh sliding window ka natural fit nahi.
- Agar negative numbers ke saath sum >= target ka shortest subarray poochha hai, simple expand/shrink window generally fail kar sakti hai. Prefix sum + monotonic deque jaise approach ki zaroorat ho sakti hai.
- Agar window ko update karne ke liye outgoing element ka effect remove nahi kar sakte, toh state design carefully check karo.
- Agar har possible subarray ka arbitrary query hai, prefix sum / segment tree / sparse table better ho sakta hai.

Interview mein 5 sawaal

- 1. Segment contiguous hai? Agar nahi, basic sliding window assume mat karo.
- 2. Window size fixed hai ya condition se decide hota hai?
- 3. Window state kya hai—sum, count, frequency, distinct count, max/min?
- 4. Right add karne aur left remove karne ka exact update kya hai?
- 5. Valid window milne par answer update karna hai ya invalid hone par?

Important warning: Positive numbers wali sum-window intuition ko negative numbers par blindly apply mat karna. Monotonicity toot jaati hai.

3. Fixed-size Sliding Window

Fixed-size window tab use hoti hai jab har candidate segment ki length exactly k ho. Pehle first k elements ka state banao. Uske baad ek step mein outgoing element remove aur incoming element add karo.

Master template — sum of every window

Java: fixed-size sum template

```
static long[] windowSums(int[] nums, int k) {
    if (nums == null || k <= 0 || k > nums.length) {
        return new long[0];
    }

    long sum = 0;
    for (int i = 0; i < k; i++) {
        sum += nums[i];
    }

    long[] ans = new long[nums.length - k + 1];
    ans[0] = sum;

    int left = 0;
    for (int right = k; right < nums.length; right++) {
        sum -= nums[left];   // remove outgoing
        left++;

        sum += nums[right];  // add incoming
        ans[left] = sum;
    }
    return ans;
}
```

Line-by-line logic

- Validation: k <= 0 ya k > n ho toh valid window exist nahi karti.
- First loop: first window ka sum ek baar O(k) mein calculate.
- Second loop: right = k se start, kyunki first k elements already included hain.
- sum -= nums[left]: old window ka leftmost element remove.
- left++: window ka start aage shift.
- sum += nums[right]: new element include.
- ans[left]: is moment par left new window ka starting index hai; answer position bhi wahi.

Dry run

nums = [4, 2, 1, 7, 8, 1, 2, 8], k = 3

Step	Outgoing	Incoming	Current window	Sum
Initial	—	4,2,1	[4,2,1]	7
1	4	7	[2,1,7]	10
2	2	8	[1,7,8]	16
3	1	1	[7,8,1]	16
4	7	2	[8,1,2]	11
5	8	8	[1,2,8]	11

Complexity: O(n) time, O(n-k+1) output space. Agar sirf maximum sum chahiye toh extra output array ki zaroorat nahi; O(1) auxiliary space.

4. Fixed Window — Maximum sum of size K

Question: given integer array and k, return maximum sum among all contiguous subarrays of exactly k elements. Sum ko long mein rakho, kyunki int overflow ho sakta hai.

Complete Java solution

```
static long maxSumSubarrayOfSizeK(int[] a, int k) {
    if (a == null || k <= 0 || k > a.length) {
        throw new IllegalArgumentException("Invalid k");
    }

    long windowSum = 0;
    for (int i = 0; i < k; i++) {
        windowSum += a[i];
    }

    long best = windowSum;

    for (int right = k; right < a.length; right++) {
        windowSum += a[right];
        windowSum -= a[right - k];
        best = Math.max(best, windowSum);
    }
    return best;
}

public static void main(String[] args) {
    int[] a = {4, 2, 1, 7, 8, 1, 2, 8};
    System.out.println(maxSumSubarrayOfSizeK(a, 3)); // 16
}
```

Alternative index formula

At right index r, outgoing index = r-k. Isliye left pointer explicitly maintain karna zaroori nahi. Ye formula fixed-size window ke liye kaafi common hai.

Compact update

```
for (int right = k; right < a.length; right++) {
    sum += a[right] - a[right - k];
    best = Math.max(best, sum);
}
```

Common mistakes

- best ko 0 se initialize karna galat ho sakta hai jab array mein saare negative numbers hon. First window sum se initialize karo.
- right-k ke bajaye right-k+1 remove karna off-by-one bug hai.
- k == n valid hai: sirf ek window hogi.
- k > n par invalid behavior define karo—empty result ya exception, problem ke contract ke mutabik.

Is pattern ke practical use

- Last k transactions ka total, fixed time bucket ka traffic, last k sensor readings ka moving sum.
- Rolling average: average = windowSum / (double) k.
- Fixed-length subarray maximum/minimum sum, average, count, product (zero handling alag).

5. Fixed Window — Average, threshold and counts

Maximum average subarray of size K

Maximum average ke liye same sum maintain hota hai. k constant hai, isliye maximum sum aur maximum average ka answer same window par aayega.

Java implementation

```
static double findMaxAverage(int[] nums, int k) {
    if (nums == null || k <= 0 || k > nums.length) {
        throw new IllegalArgumentException("Invalid k");
    }

    long sum = 0;
    for (int i = 0; i < k; i++) sum += nums[i];

    long best = sum;
    for (int right = k; right < nums.length; right++) {
        sum += nums[right] - nums[right - k];
        best = Math.max(best, sum);
    }
    return best / (double) k;
}
```

Count windows whose average is at least threshold

Average >= threshold ko sum >= threshold*k mein convert kar sakte ho. Isse floating-point comparison avoid hota hai.

Java implementation

```
static int countWindowsAverageAtLeast(int[] a, int k, int threshold) {
    if (a == null || k <= 0 || k > a.length) return 0;

    long sum = 0;
    for (int i = 0; i < k; i++) sum += a[i];

    int count = sum >= (long) threshold * k ? 1 : 0;

    for (int right = k; right < a.length; right++) {
        sum += a[right] - a[right - k];
        if (sum >= (long) threshold * k) count++;
    }
    return count;
}
```

Fixed window mein count/frequency maintain karna

Sum ke alawa aap window ke andar even numbers, vowels, zeros, ya kisi category ke items ka count bhi maintain kar sakte ho.

Example: each window mein even numbers count

```
static int[] evenCounts(int[] a, int k) {
    if (a == null || k <= 0 || k > a.length) return new int[0];

    int count = 0;
    for (int i = 0; i < k; i++) {
        if (a[i] % 2 == 0) count++;
    }

    int[] ans = new int[a.length - k + 1];
    ans[0] = count;

    for (int right = k; right < a.length; right++) {
        if (a[right - k] % 2 == 0) count--;
        if (a[right] % 2 == 0) count++;
        ans[right - k + 1] = count;
    }
    return ans;
}
```

Reusable principle: jo property window ke elements se incrementally update ho sakti hai, usko fixed window mein maintain kiya ja sakta hai.

6. Fixed Window — Frequency array and anagram matching

Strings mein fixed window ka powerful use character frequency compare karna hai. Example: pattern p ke anagrams s mein kahan-kahan appear hote hain? Har candidate substring ka sort karna expensive hai. Frequency counts maintain karo.

Frequency array kab?

- Input lowercase English letters tak limited ho toh int[26] simple aur fast hai.
- All ASCII characters ke liye int[128] ya int[256] use kar sakte ho.
- Unicode/general character set ya arbitrary keys ke liye HashMap use karo.

Find all anagram start indices — lowercase letters

```
static List<Integer> findAnagrams(String s, String p) {
    List<Integer> ans = new ArrayList<>();
    if (s == null || p == null || p.length() > s.length()) return ans;

    int k = p.length();
    int[] need = new int[26];
    int[] have = new int[26];

    for (int i = 0; i < k; i++) {
        need[p.charAt(i) - 'a']++;
        have[s.charAt(i) - 'a']++;
    }

    if (Arrays.equals(need, have)) ans.add(0);

    for (int right = k; right < s.length(); right++) {
        have[s.charAt(right - k) - 'a']--;
        have[s.charAt(right) - 'a']++;

        if (Arrays.equals(need, have)) {
            ans.add(right - k + 1);
        }
    }
    return ans;
}
```

Imports: java.util.*; Complexity: code above mein Arrays.equals har window par 26 counters compare karta hai, so O(26*n) = O(n) for fixed alphabet. Space O(26).

Optimization idea: matched counter

Agar alphabet bahut bada ho, har step par poori frequency array compare karna mehenga ho sakta hai. Ek matched/difference counter maintain karke O(1) updates possible hain, lekin logic tricky hai. Pehle simple correct implementation master karo.

Dry run intuition

s = 'cbaebabacd', p = 'abc'. Need frequency: a=1,b=1,c=1. First window 'cba' matches. Window shift karte waqt outgoing char ka count decrement aur incoming ka increment hota hai. Agar resulting frequencies equal, current substring anagram hai.

7. Variable-size Sliding Window

Variable window mein size fixed nahi hota. Right pointer elements add karke window expand karta hai. Jab condition invalid ho jaati hai, left pointer move karke elements remove karta hai. Kabhi answer valid window par update hota hai; kabhi condition ke invalid hone ke baad.

Generic template

Java: variable window skeleton

```
int left = 0;
for (int right = 0; right < n; right++) {
    // 1) Include nums[right] in the window.
    add(nums[right]);

    // 2) While the window violates the rule, shrink it.
    while (windowIsInvalid()) {
        remove(nums[left]);
        left++;
    }

    // 3) Window is valid here. Update answer if this
    //    problem asks for longest valid window.
    answer = Math.max(answer, right - left + 1);
}
```

Yahan add(), remove() aur windowIsInvalid() conceptual placeholders hain; actual code mein sum/count/map update likhna hota hai. Har problem mein inka exact meaning alag hota hai.

Longest vs shortest: answer kab update karein?

Question type	Typical answer update	Why
Longest valid window	Shrink until valid, then update	Current valid window maximum length candidate hai.
Shortest window meeting target	While condition satisfied, update then shrink	Har valid window ko chhota karne ki koshish.
Count of valid subarrays	Often add right-left+1 after valid	Fixed right ke saath ending valid subarrays count hote hain.
Exactly K distinct	AtMost(K)-AtMost(K-1)	Direct exact count awkward; difference technique.

Important condition: monotonicity

Positive numbers ke sum ke case mein right add karne se sum badhta hai; left remove karne se sum ghatta hai. Is predictable behavior se threshold window kaam karti hai. Negative numbers aane par sum kabhi badh bhi sakta hai, ghat bhi sakta hai—simple two-pointer condition reliable nahi rehti.

8. Variable Window — Longest substring without repeating characters

Question: string mein longest substring ki length find karo jisme koi character repeat na ho. Window valid tab hai jab har character ka frequency <= 1 ho.

Approach

- HashSet use karke check karo ki incoming character already window mein hai ya nahi.
- Agar duplicate hai, left side se characters remove karte jao jab tak duplicate hat na jaye.
- Window valid hone ke baad length update karo.

Java: HashSet approach

```
static int lengthOfLongestSubstring(String s) {
    if (s == null) return 0;

    Set<Character> set = new HashSet<>();
    int left = 0;
    int best = 0;

    for (int right = 0; right < s.length(); right++) {
        char current = s.charAt(right);

        while (set.contains(current)) {
            set.remove(s.charAt(left));
            left++;
        }

        set.add(current);
        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

Dry run: 'abcabcbb'

right char	Action	Current window	best
a	add a	a	1
b	add b	ab	2
c	add c	abc	3
a	remove a; add a	bca	3
b	remove b; add b	cab	3
c	remove c; add c	abc	3
b	remove a,b; add b	cb	3
b	remove c,b; add b	b	3

Last-seen index optimization

HashMap/array mein character ka last seen index store karo. Duplicate milne par left ko seedha aage jump kara sakte ho. left ko kabhi peeche nahi le jaana: left = max(left, lastSeen+1).

Java: last-seen index

```
static int longestUniqueSubstring(String s) {
    int[] last = new int[Character.MAX_VALUE + 1];
    Arrays.fill(last, -1);

    int left = 0, best = 0;
    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
```

```
            left = Math.max(left, last[c] + 1);
            last[c] = right;
            best = Math.max(best, right - left + 1);
        }
        return best;
    }
```

Complexity: O(n) time, O(character-set size) space. Java char UTF-16 code unit hai; full Unicode code points ke liye codePointAt / code-point iteration ka alag treatment chahiye.

9. Variable Window — Longest substring with at most K distinct characters

Window valid hai jab distinct characters ki count <= k ho. HashMap character ki frequency store karta hai. Frequency zero hone par key remove karni zaroori hai, warna map.size() distinct count nahi batayega.

Java implementation

```
static int longestAtMostKDistinct(String s, int k) {
    if (s == null || k <= 0) return 0;

    Map<Character, Integer> freq = new HashMap<>();
    int left = 0, best = 0;

    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        freq.put(c, freq.getOrDefault(c, 0) + 1);

        while (freq.size() > k) {
            char out = s.charAt(left);
            int count = freq.get(out) - 1;
            if (count == 0) freq.remove(out);
            else freq.put(out, count);
            left++;
        }

        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

Example

s = 'eceba', k = 2. Valid longest substring 'ece' hai, length 3. Jab 'b' add hota hai, distinct count 3 ho jaata hai. Left move karke 'e' ki purani occurrence aur phir 'c' remove hota hai; valid window dobara banne par answer update hota hai.

When to use HashMap vs int[26]

Situation	Choose	Reason
Only 'a' to 'z'	int[26]	Less overhead, simple index.
ASCII chars	int[128] / int[256]	Fixed-size array works.
Any Java char	int[65536] or Map	Array can be large; map is flexible.
Words/categories/IDs	HashMap	Keys are not limited to characters.

Bug alert: map se key ko tabhi remove karo jab frequency 0 ho. Sirf value 0 rakhne se map.size() distinct count galat de sakta hai.

10. At Most K aur Exactly K

Exactly K distinct characters/subarrays ka count direct maintain karna tricky ho sakta hai. Standard trick: exactly(K) = atMost(K) - atMost(K-1). Ye tab use hoti hai jab at-most count easy ho.

At most K distinct subarrays count

Agar current window [left..right] mein at most K distinct values hain, toh fixed right ke liye ye saare subarrays valid hain: [right..right], [right-1..right], ..., [left..right]. Count = right-left+1.

Java: count subarrays with at most K distinct

```
static long subarraysAtMostKDistinct(int[] a, int k) {
    if (k < 0) return 0;

    Map<Integer, Integer> freq = new HashMap<>();
    int left = 0;
    long total = 0;

    for (int right = 0; right < a.length; right++) {
        freq.put(a[right], freq.getOrDefault(a[right], 0) + 1);

        while (freq.size() > k) {
            int x = a[left++];
            int count = freq.get(x) - 1;
            if (count == 0) freq.remove(x);
            else freq.put(x, count);
        }

        total += right - left + 1;
    }
    return total;
}

static long subarraysExactlyKDistinct(int[] a, int k) {
    if (k <= 0) return 0;
    return subarraysAtMostKDistinct(a, k)
         - subarraysAtMostKDistinct(a, k - 1);
}
```

Why add right-left+1?

Example: current valid window [1,2,1] ending at right=2, left=0. Ending at index 2 wale valid subarrays: [1,2,1], [2,1], [1]. Total 3 = 2-0+1.

Exactly K distinct — dry run

a = [1,2,1,2,3], K=2. Exactly two distinct values wale subarrays ka answer atMost(2) minus atMost(1) hai. Is formula se har subarray individually enumerate karne ki zaroorat nahi.

Do not confuse: “Longest substring with exactly K distinct” aur “count subarrays with exactly K distinct” alag output problems hain. Longest ke liye length track hoti hai; count ke liye right-left+1 add hota hai.

11. Smallest subarray with sum at least target

Positive integers ke array mein minimum length subarray find karo jiska sum >= target. Right add karo; jab sum target tak pahunch jaaye, answer update karo aur left se shrink karte raho.

Java implementation — positive numbers

```
static int minSubArrayLen(int target, int[] nums) {
    int left = 0;
    long sum = 0;
    int best = Integer.MAX_VALUE;

    for (int right = 0; right < nums.length; right++) {
        sum += nums[right];

        while (sum >= target) {
            best = Math.min(best, right - left + 1);
            sum -= nums[left];
            left++;
        }
    }
    return best == Integer.MAX_VALUE ? 0 : best;
}
```

Why update inside while?

Jab sum >= target hai, current window valid hai. Iske baad left remove karne se window chhoti hogi. Har valid shrinking step par length check karna zaroori hai; pehla valid window necessarily shortest nahi hota.

Negative numbers ka warning

Agar array mein negative values allowed hain, simple sliding window fail ho sakti hai. Example: negative value add karne se sum kam bhi ho sakta hai; left remove karne se sum badh bhi sakta hai. Isliye monotonic rule nahi bachta. General integer case ke liye prefix sums + monotonic deque (shortest subarray with sum at least K) ek common solution hai.

Useful applications

- Minimum number of consecutive days to reach cumulative sales threshold (agar values non-negative).
- Smallest continuous batch whose total workload reaches a target.
- Positive weights/costs ke contiguous group ka shortest qualifying segment.

12. Longest subarray with sum at most / at most constraint

Agar values non-negative hain, sum <= limit wali longest window maintain karna possible hai: right add karo; sum > limit ho toh left shrink karo; valid hone par length update.

Java implementation — non-negative array

```
static int longestSumAtMost(int[] a, long limit) {
    int left = 0, best = 0;
    long sum = 0;

    for (int right = 0; right < a.length; right++) {
        sum += a[right];

        while (left <= right && sum > limit) {
            sum -= a[left++];
        }

        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

Ye implementation non-negative numbers ke liye assume karti hai. Negative numbers ke saath same logic reliable nahi.

Longest ones after flipping at most K zeros

Binary array mein maximum consecutive 1s find karo agar at most k zeros ko 1 bana sakte ho. Window ke andar zero count <= k rakho.

Java implementation

```
static int longestOnes(int[] nums, int k) {
    int left = 0, zeros = 0, best = 0;

    for (int right = 0; right < nums.length; right++) {
        if (nums[right] == 0) zeros++;

        while (zeros > k) {
            if (nums[left] == 0) zeros--;
            left++;
        }

        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

Dry run

nums = [1,1,1,0,0,0,1,1,1,1,0], k=2. Window mein maximum 2 zeros allow hain. Third zero enter kare toh left ko move karke ek zero remove karo. Valid window ke baad length update hoti hai.

13. Character replacement — longest repeating character replacement

Question: uppercase string mein at most k characters replace karke longest substring banao jisme saare characters same hon. Window valid hai agar windowLength - maxFrequency <= k.

Formula

Required replacements = window length − window ke most frequent character ki frequency.
Agar required replacements <= k, window feasible hai.

Java implementation

```
static int characterReplacement(String s, int k) {
    int[] freq = new int[26];
    int left = 0, maxFreq = 0, best = 0;

    for (int right = 0; right < s.length(); right++) {
        int idx = s.charAt(right) - 'A';
        freq[idx]++;
        maxFreq = Math.max(maxFreq, freq[idx]);

        while ((right - left + 1) - maxFreq > k) {
            freq[s.charAt(left) - 'A']--;
            left++;
        }

        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

maxFreq ko shrink par recalculate kyun nahi kiya?

Is optimized known solution mein maxFreq stale (current window ke actual maximum se bada) reh sakta hai. Isse best length ka answer phir bhi correct milta hai, kyunki stale value sirf window ko temporarily zyada permissive rakhti hai; standard proof ke mutabik best length valid candidate se establish hoti hai. Lekin agar tumhe har moment exact current window validity chahiye, frequencies se max dobara calculate karo (26 letters ke liye O(26) per step) ya suitable structure use karo.

Input assumptions

- Ye code uppercase A-Z assume karta hai. Lowercase ya arbitrary Unicode ke liye indexing badlo / HashMap use karo.
- k < 0 invalid input hai; production code mein validate karo.

14. Minimum window substring

String s mein shortest substring find karo jisme t ke saare required characters required multiplicity ke saath present hon. Ye classic hard-level variable window problem hai.

Idea

- need map: t mein har char kitni baar chahiye.
- window map: current window mein har char kitni baar hai.
- formed/valid count: kitne unique required characters ki frequency requirement meet hui.
- Right expand karke requirement complete karo; jab complete ho, left shrink karke shortest answer improve karo.

Java implementation

```
static String minWindow(String s, String t) {
    if (s == null || t == null || t.isEmpty() || s.length() < t.length()) {
        return "";
    }

    Map<Character, Integer> need = new HashMap<>();
    for (char c : t.toCharArray()) {
        need.put(c, need.getOrDefault(c, 0) + 1);
    }

    Map<Character, Integer> have = new HashMap<>();
    int required = need.size();
    int formed = 0;
    int left = 0;
    int bestLen = Integer.MAX_VALUE;
    int bestStart = 0;

    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        have.put(c, have.getOrDefault(c, 0) + 1);

        if (need.containsKey(c)
                && have.get(c).intValue() == need.get(c).intValue()) {
            formed++;
        }

        while (formed == required) {
            int len = right - left + 1;
            if (len < bestLen) {
                bestLen = len;
                bestStart = left;
            }

            char out = s.charAt(left);
            have.put(out, have.get(out) - 1);

            if (need.containsKey(out)
                    && have.get(out) < need.get(out)) {
                formed--;
            }
            left++;
        }
    }

    return bestLen == Integer.MAX_VALUE
            ? "" : s.substring(bestStart, bestStart + bestLen);
}
```

Important detail

Repeated characters matter. Agar t = 'AABC', toh window mein A kam se kam 2 baar, B ek baar aur C ek baar hona chahiye. Sirf distinct character presence check karna enough nahi.

Complexity: expected O(|s| + |t|) time with HashMap, O(character set) auxiliary space.

15. Monotonic Deque — maximum of every window

Har size-k window ka maximum nikalna ho toh har window ko scan karna O(nk) hoga. Deque mein indices rakho, values ko decreasing order mein maintain karo. Front par current window ka maximum index rahega.

Deque ke rules

- Right index add karne se pehle back se un indices ko remove karo jinki values current value se <= hain. Woh future windows mein kabhi maximum nahi banenge.
- Front index agar window ke left boundary se bahar hai, remove karo.
- Current window ready hone par front value answer hai.

Java: sliding window maximum

```
static int[] maxSlidingWindow(int[] nums, int k) {
    if (nums == null || k <= 0 || k > nums.length) return new int[0];

    int n = nums.length;
    int[] ans = new int[n - k + 1];
    Deque<Integer> dq = new ArrayDeque<>();

    for (int right = 0; right < n; right++) {
        // Remove indices outside the window.
        while (!dq.isEmpty() && dq.peekFirst() <= right - k) {
            dq.pollFirst();
        }

        // Keep values decreasing from front to back.
        while (!dq.isEmpty()
                && nums[dq.peekLast()] <= nums[right]) {
            dq.pollLast();
        }

        dq.offerLast(right);

        // First complete window ends at k - 1.
        if (right >= k - 1) {
            ans[right - k + 1] = nums[dq.peekFirst()];
        }
    }
    return ans;
}
```

Dry run

nums = [1,3,-1,-3,5,3,6,7], k=3. Output = [3,3,5,5,6,7]. Deque indices store candidate maxima, not all window values. Each index enters once and exits at most once.

Complexity

O(n) time, O(k) deque space. Har index maximum ek baar add aur ek baar remove hota hai—isi wajah se total work linear hai.

16. Monotonic Deque — minimum and max-min bounded window

Minimum of every window

Maximum wale code mein deque ko increasing values order mein maintain karo: back se larger/equal values remove karo. Front par minimum rahega.

Core change for minimum

```
while (!dq.isEmpty()
        && nums[dq.peekLast()] >= nums[right]) {
    dq.pollLast();
}
```

Longest subarray where max - min <= limit

Do deques maintain karo: maxDeque decreasing values, minDeque increasing values. Jab max-min > limit ho, left shrink karo aur dono deques se expired indices remove karo.

Java implementation

```
static int longestSubarrayWithinLimit(int[] nums, int limit) {
    Deque<Integer> maxD = new ArrayDeque<>();
    Deque<Integer> minD = new ArrayDeque<>();
    int left = 0, best = 0;

    for (int right = 0; right < nums.length; right++) {
        while (!maxD.isEmpty()
                && nums[maxD.peekLast()] < nums[right]) {
            maxD.pollLast();
        }
        maxD.offerLast(right);

        while (!minD.isEmpty()
                && nums[minD.peekLast()] > nums[right]) {
            minD.pollLast();
        }
        minD.offerLast(right);

        while ((long) nums[maxD.peekFirst()]
                - nums[minD.peekFirst()] > limit) {
            left++;
            if (maxD.peekFirst() < left) maxD.pollFirst();
            if (minD.peekFirst() < left) minD.pollFirst();
        }

        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

Where used?

- Streaming telemetry: recent k readings ka max/min.
- Market/price range: segment ka fluctuation limit ke andar rakhna.
- Signal processing: rolling extrema.
- Queue scheduling: recent priority/latency extrema.

17. Sliding Window + Hashing: count subarrays with all distinct values

Agar question poochhe har window ya subarray mein duplicates kitne hain, frequency map / set useful hai. Example: longest subarray with all unique integers, string ke unique-character version jaisa hi pattern.

Java: longest subarray with all distinct integers

```
static int longestAllDistinct(int[] a) {
    Set<Integer> set = new HashSet<>();
    int left = 0, best = 0;

    for (int right = 0; right < a.length; right++) {
        while (set.contains(a[right])) {
            set.remove(a[left]);
            left++;
        }
        set.add(a[right]);
        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

Frequency map template — reusable

Add/remove helper methods

```
static void add(Map<Integer, Integer> freq, int x) {
    freq.put(x, freq.getOrDefault(x, 0) + 1);
}

static void remove(Map<Integer, Integer> freq, int x) {
    int count = freq.get(x) - 1;
    if (count == 0) freq.remove(x);
    else freq.put(x, count);
}
```

In helpers ko tab use karo jab multiple problems mein same frequency bookkeeping repeat ho. Interview mein pehle inline code likhna often clearer hota hai; helper abstraction tab banao jab repeated logic ho.

Data structure selection

Need	Structure	Typical operation
Presence only	HashSet	contains / add / remove
Frequency per key	HashMap	getOrDefault / put / remove
Small fixed alphabet	int[26]	counter increment/decrement
Window maximum/minimum	ArrayDeque	front/back operations
Sum/count	long/int variable	add incoming, subtract outgoing

18. Product of window / product less than K

Positive integers ke array mein product < k wale contiguous subarrays count karne ke liye variable window use ho sakti hai. Product right add karne par non-decreasing hota hai, aur left remove karne par divide hota hai—ye positivity par depend karta hai.

Java implementation — positive integers

```
static long numSubarrayProductLessThanK(int[] nums, int k) {
    if (k <= 1) return 0;

    long product = 1;
    int left = 0;
    long count = 0;

    for (int right = 0; right < nums.length; right++) {
        product *= nums[right];

        while (product >= k && left <= right) {
            product /= nums[left];
            left++;
        }

        count += right - left + 1;
    }
    return count;
}
```

Important assumptions and overflow

- All values positive integers hon. Zero/negative values ke liye ye direct logic valid nahi.
- Product long mein bhi overflow kar sakta hai agar constraints huge hon. Problem constraints check karo; multiplication se pehle product > (k-1)/x type guard ya alternative strategy consider karo.
- k <= 1 ke liye positive products < k possible nahi, isliye 0.

Why count = right-left+1?

Current valid window ke ending-at-right saare suffixes valid hain: [right..right], [right-1..right], ... [left..right]. Count = right-left+1.

19. Two-dimensional sliding window

2D matrix mein sliding window row/column direction mein apply ho sakti hai. Lekin 2D problems ke liye pehle clarify karo: fixed rectangle ka sum, maximum rectangle, ya har row ke horizontal windows? Ek common method: har row par 1D fixed window apply karna.

Horizontal window sum for each row

Java: each row ke har k-column window ka sum

```
static int[][] horizontalWindowSums(int[][] a, int k) {
    int rows = a.length;
    int cols = a[0].length;
    if (k <= 0 || k > cols) return new int[0][0];

    int[][] ans = new int[rows][cols - k + 1];

    for (int r = 0; r < rows; r++) {
        int sum = 0;
        for (int c = 0; c < k; c++) sum += a[r][c];
        ans[r][0] = sum;

        for (int c = k; c < cols; c++) {
            sum += a[r][c] - a[r][c - k];
            ans[r][c - k + 1] = sum;
        }
    }
    return ans;
}
```

Fixed rectangle sum queries

Agar arbitrary rectangle ka sum poochha hai, 2D prefix sum usually better hai. Agar har possible rectangle ko slide karke process kar rahe ho, row-wise/column-wise rolling sums useful ho sakte hain.

Example use

- Image processing: k×k local average / blur filters.
- Grid sensor data: fixed-size neighborhood sum.
- Matrix mein each row ke rolling totals.

20. Streaming / real-world implementation

Sliding Window sirf coding interview trick nahi. Jab data continuously aata hai aur hume recent N records ka state chahiye, tab rolling window natural fit hoti hai.

Real use-case	Window state	How update works
Website requests per last 60 seconds	Request count	New request add; timestamps older than 60s expire.
CPU utilization last 5 minutes	Rolling average	Add new sample, subtract sample leaving time window.
Sensor readings	Recent max/min	Monotonic deque of indices/timestamps.
Fraud detection	Count/frequency of events	Map event category counts within time window.
Network packet monitoring	Bytes in last interval	Queue timestamps + rolling byte sum.

Time-based window vs count-based window

Count-based window: exactly last k items. Time-based window: all items whose timestamps fall inside [now-duration, now]. Time-based window mein items irregular intervals par aate hain, isliye queue/deque mein timestamps store karna hota hai.

Illustrative Java pattern: time-based rolling count

```
class TimeWindowCounter {
    private final long durationMillis;
    private final Deque<Long> timestamps = new ArrayDeque<>();

    TimeWindowCounter(long durationMillis) {
        this.durationMillis = durationMillis;
    }

    void record(long nowMillis) {
        timestamps.addLast(nowMillis);
        evictOld(nowMillis);
    }

    int count(long nowMillis) {
        evictOld(nowMillis);
        return timestamps.size();
    }

    private void evictOld(long nowMillis) {
        long cutoff = nowMillis - durationMillis;
        while (!timestamps.isEmpty()
                && timestamps.peekFirst() <= cutoff) {
            timestamps.pollFirst();
        }
    }
}
```

Production note: concurrent access, clock choice, out-of-order events, memory limits aur exact boundary (<= cutoff vs < cutoff) explicitly define karna hota hai. Upar ka snippet single-threaded illustrative implementation hai.

21. Brute Force se Sliding Window tak kaise socho

Example: max sum of k consecutive values

Brute force: har start index se k elements ka sum calculate. Approximately (n-k+1)*k work, O(nk). Optimization: adjacent windows overlap karti hain; sirf ek element nikalta hai aur ek aata hai. Isliye old sum update karo: newSum = oldSum - outgoing + incoming.

Approach	Time	Extra space	When useful
Nested loops	O(nk)	O(1)	Small n, simplest baseline.
Rolling sum window	O(n)	O(1)	Fixed k, sum/aggregate incrementally update ho.
Prefix sum	O(n) build; O(1) per query	O(n)	Many arbitrary range-sum queries.
Deque	O(n)	O(k)	Every window ka max/min chahiye.

Optimization ka interview narration

- 1. Pehle brute force explain karo: har possible contiguous window evaluate karunga.
- 2. Observe karo ki consecutive windows overlap karti hain.
- 3. Repeated work identify karo: k-1 elements common hain.
- 4. State choose karo: sum/frequency/deque.
- 5. Add/remove updates define karo.
- 6. Pointer movement prove karo: each element limited number of times process hota hai.
- 7. Complexity compare karo.

Amortized O(n) ka intuition

Variable window mein nested while loop dekhkar $O(n^2)$ assume mat karo. left aur right dono sirf aage move karte hain. right n baar increment hota hai; left maximum n baar increment hota hai. Total pointer moves O(2n) = O(n).

22. Java implementation toolkit

Imports

Common imports

```
import java.util.*;
```

Useful Java methods

Method	Meaning	Use
map.getOrDefault(key, 0)	Key absent ho toh default 0	Frequency increment.
map.remove(key)	Key delete	Frequency zero hone par.
set.contains(x)	Presence check	Duplicate detection.
deque.peekFirst()	Front element without removal	Current max/min index.
deque.pollFirst()	Remove front	Expired index.
deque.peekLast()	Back element without removal	Monotonic cleanup.
deque.pollLast()	Remove back	Dominated candidate.
deque.offerLast(x)	Add at back	New index insert.

Integer overflow

- Array elements ka sum potentially n * maxValue ho sakta hai. `long sum` use karo.
- Count of subarrays maximum n(n+1)/2 ho sakta hai. `long count` use karo.
- Product window mein overflow ka risk specially high hai.

Inclusive/exclusive boundary

Agar window inclusive [left,right] hai, length = right-left+1. Agar half-open [left,right) hai, length = right-left. Is handbook ke code mein mostly inclusive boundaries use ki gayi hain.

Null and invalid inputs

Interview question ka contract follow karo. Kuch platforms guarantee dete hain input non-null aur k valid hai. Production-quality code mein null, empty array, k<=0 aur k>n ka behavior define karo.

23. Common mistakes and debugging checklist

Mistake	Symptom	Fix
Wrong outgoing index	Window sum off by one	Fixed window: outgoing = right-k.
Update answer too early	Incomplete first window counted	Fixed window: right >= k-1 ke baad answer.
Shrink with if instead of while	Window still invalid	Condition false hone tak while.
Map key zero par retain	Distinct count wrong	Frequency zero par remove key.
Best = 0 for max sum	All-negative array wrong	First window sum se initialize.
Negative numbers with sum window	Unexpected wrong answer	Prefix sum / deque / other approach.
Character indexing assumption wrong	ArrayIndexOutOfBounds	Input alphabet validate or Map.
Deque values store, indices nahi	Cannot expire old duplicates safely	Store indices.
Wrong count formula	Subarray count wrong	At-most valid window: right-left+1.
int overflow	Large test cases fail	Use long where bounds require.

Before submitting, test these cases

- Empty array/string.
- One element; k=1; k=n; k>n; k=0.
- All values equal.
- All values distinct.
- All negative values for max-sum fixed window.
- Zeros and duplicates.
- Answer at first window and answer at last window.
- Window must shrink multiple times in a row.
- Threshold never reached / every window valid.
- Large input values that can overflow int.

24. Problem catalogue — fixed-size patterns

Problem	Pattern / state	Difficulty
Maximum sum subarray of size K	Rolling sum	Easy
Average of every window of size K	Rolling sum / division	Easy
Count windows with average >= threshold	Rolling sum + threshold*k	Easy
Maximum number of vowels in substring of length K	Rolling vowel count	Easy
Find all anagrams in a string	Frequency array + fixed window	Medium
Permutation in string	Frequency match + fixed window	Medium
Sliding window maximum	Monotonic decreasing deque	Hard
Sliding window minimum	Monotonic increasing deque	Hard
First negative number in every window	Deque of candidate indices/values	Medium
K-radius averages	Prefix/rolling sum	Medium

Hints for selected questions

- Maximum vowels: first k chars mein vowels count; shift par outgoing vowel subtract, incoming vowel add; max maintain.
- Permutation in string: pattern length fixed; current window frequency pattern ke equal ho toh permutation present.
- First negative in every window: negative element ke indices queue mein rakho; expired index remove; front first negative.
- K-radius averages: radius k ka window size 2k+1; rolling sum maintain; boundaries par -1 return.

Implementation exercise: maximum vowels

Java implementation

```
static int maxVowels(String s, int k) {
    int count = 0;
    for (int i = 0; i < k; i++) {
        if (isVowel(s.charAt(i))) count++;
    }

    int best = count;
    for (int right = k; right < s.length(); right++) {
        if (isVowel(s.charAt(right - k))) count--;
        if (isVowel(s.charAt(right))) count++;
        best = Math.max(best, count);
    }
    return best;
}

static boolean isVowel(char c) {
    return c=='a' || c=='e' || c=='i' || c=='o' || c=='u';
}
```

Assumption: s length >= k and k valid. Robust version mein input validation add karo.

25. Problem catalogue — variable-size patterns

Problem	Pattern / idea	Difficulty
Longest substring without repeating characters	Set / last-seen index	Medium
Longest substring with at most K distinct	Frequency map	Medium
Subarrays with K different integers	AtMost(K)-AtMost(K-1)	Hard
Minimum size subarray sum	Positive sum expand/shrink	Medium
Max consecutive ones III	Count zeros <= K	Medium
Longest repeating character replacement	windowLen - maxFreq <= K	Medium
Minimum window substring	need/have + formed	Hard
Fruit into baskets	At most 2 distinct	Medium
Max subarray with at most K zeros	Frequency/zero count	Medium
Subarray product less than K	Positive product window	Medium
Longest continuous subarray with abs diff <= limit	Two monotonic deques	Hard

Problem ko template se map karne ka practice

Har question ke liye notebook mein ye 6 lines likho: (1) window definition, (2) valid condition, (3) add operation, (4) remove operation, (5) answer update position, (6) data structure. Jab ye clear ho jaaye tab code likho.

Code-writing exercise

- Pehle brute force likho, chahe $O(n^2)$ ho.
- Ek example manually dry run karo.
- Window state aur invalid condition define karo.
- Java code likho; variable names meaningful rakho.
- Test edge cases aur constraints.

26. Advanced variations and recognition

Window ke andar multiple statistics

Ek hi window mein sum, zero count, distinct count aur max/min maintain karna possible hai. Lekin har state ka update independent aur correct hona chahiye. Complexity total update cost se decide hogi.

Two windows / two pointers

Kuch problems mein two windows maintain hoti hain, jaise exactly K distinct via two at-most counts. Har window ka apna left pointer aur frequency map ho sakta hai.

Binary search + sliding window

Kabhi answer length par binary search karke check karte hain ki given length ka koi window condition satisfy karta hai ya nahi. Ye tab useful hai jab feasibility check monotonic ho. Lekin direct sliding window possible ho toh unnecessary binary search mat add karo.

Circular array

Circular array mein end ke baad start aata hai. Common trick array ko conceptually double length treat karna, lekin window length n se zyada na hone dena. Modulo indexing use ho sakti hai.

Circular fixed window — conceptual indexing

```
int n = a.length;
for (int step = 0; step < n + k - 1; step++) {
    int value = a[step % n];
    // rolling update; ensure no window length exceeds n
}
```

Ye snippet sirf indexing idea hai, complete problem solution nahi. Exact bounds question ke hisaab se define karo.

Streaming data

Agar data ka total size unknown hai, fixed-size rolling sum/queue use karke O(k) memory mein recent k items maintain kar sakte ho. Agar time-window hai toh timestamps ke basis par expired items remove karo.

27. Interview-ready templates — revision page

Template A: fixed-size rolling sum

Remember

```
sum = sum(first k elements)
answer = evaluate(sum)

for right = k to n-1:
    sum += a[right]       // incoming
    sum -= a[right-k]     // outgoing
    answer = evaluate(sum)
```

Template B: longest valid variable window

Remember

```
left = 0
for right = 0 to n-1:
    add(a[right])

    while (window invalid):
        remove(a[left])
        left++

    answer = max(answer, right-left+1)
```

Template C: shortest valid variable window

Remember

```
left = 0
for right = 0 to n-1:
    add(a[right])

    while (window valid):
        answer = min(answer, right-left+1)
        remove(a[left])
        left++
```

Template D: count valid subarrays ending at right

Remember

```
left = 0
for right = 0 to n-1:
    add(a[right])

    while (window invalid):
        remove(a[left])
        left++

    count += right-left+1
```

Template E: monotonic deque

Remember

```
remove expired indices from front
while back candidate is dominated by current:
    remove back
add current index at back
answer = value at front (once window is full)
```

Template warning: Ye skeletons hain, blindly paste karne wale solutions nahi. `window invalid`, `add`, `remove`, aur answer update problem ke exact statement se derive karo.

28. Formula and complexity cheat sheet

Pattern	Formula / condition	Time
Window length	right-left+1	O(1)
Fixed sum shift	newSum = oldSum - outgoing + incoming	O(n) total
Fixed average	sum / (double) k	O(n) total
Longest valid window	max(right-left+1) after shrink	O(n) if pointers monotonic
Shortest valid window	min(right-left+1) while valid	O(n) if pointers monotonic
At most K count	add right-left+1	O(n) expected with HashMap
Exactly K count	atMost(K)-atMost(K-1)	O(n) expected
Min subarray sum >= target	positive values only: shrink while sum>=target	O(n)
Longest unique substring	shrink while duplicate / jump left by lastSeen	O(n) expected
Window maximum	decreasing deque of indices	O(n)
Window minimum	increasing deque of indices	O(n)
Max-min <= limit	two monotonic deques	O(n)
Product < K	positive values only; shrink while product>=K	O(n)

Complexity ka proof kaise likhein

- Fixed rolling window: each element once initial/add and at most once outgoing => O(n).
- Variable window: right n times, left at most n times => O(n), assuming O(1) expected map/set operations.
- Deque: each index inserted once and removed at most once from either end => O(n).
- Space: map/set/deque mein maximum current window ke items => O(n) worst case; fixed alphabet array => O(1) relative to n.

29. 14-day practice roadmap

Day	Focus	Target task
1	Window basics	Fixed-size sum and average, 4 dry runs.
2	Fixed window variations	Even count, vowel count, threshold windows.
3	Frequency arrays	Anagram indices and permutation check.
4	Variable window basics	Longest unique substring using Set.
5	Frequency map	At most K distinct, fruit baskets.
6	Counting	At most K and exactly K subarrays.
7	Positive sum windows	Minimum size subarray sum.
8	Binary window	Max consecutive ones III.
9	Replacement	Longest repeating character replacement.
10	Hard string window	Minimum window substring.
11	Deque basics	Sliding window maximum/minimum.
12	Two deques	Longest subarray max-min <= limit.
13	Mixed practice	Identify pattern without seeing tags.
14	Mock interview	Explain, code, dry run, complexity, edge cases.

Daily solving method

- 5 minutes: statement ko apne words mein likho.
- 5 minutes: brute force approach and complexity.
- 10 minutes: window definition + state + valid/invalid condition.
- 15–25 minutes: implementation and dry run.
- 5 minutes: edge cases and complexity proof.

30. Final self-test — kya tum ready ho?

- 1. Fixed window mein first complete window kab milti hai?
- 2. `right-k` outgoing index kyun hai?
- 3. Variable window mein `if` ke badle `while` kab zaroori hai?
- 4. Longest valid window mein answer shrink ke baad kyun update hota hai?
- 5. Shortest valid window mein answer while ke andar kyun update hota hai?
- 6. `atMost(K)-atMost(K-1)` exactly K kyun deta hai?
- 7. Frequency zero hone par map key remove kyun karte hain?
- 8. Sliding window maximum mein deque values ke badle indices kyun store karte hain?
- 9. Negative numbers ke saath sum >= target window kyun fail kar sakti hai?
- 10. O(n) amortized proof mein left pointer kitni baar move kar sakta hai?

Answer key — short

- 1. right == k-1 par.
- 2. Current window size k hai; right-k old leftmost index hota hai.
- 3. Jab tak window invalid hai, multiple left moves needed ho sakte hain.
- 4. Shrink ke baad window valid hoti hai; longest candidate tab evaluate hota hai.
- 5. Har valid window ko shrink karke shorter candidate dhoondhna hota hai.
- 6. At most K mein 0..K distinct ke counts hain; at most K-1 subtract karne se exactly K bachta hai.
- 7. Map size ko distinct key count ke roop mein use karne ke liye.
- 8. Expiry boundary check karne ke liye indices chahiye.
- 9. Add/remove se sum ka predictable monotonic behavior nahi rehta.
- 10. Maximum n times; left kabhi peeche nahi jaata.

Final rule

Sliding Window master karne ka real test: Tum bina pattern name dekhe problem ko read karke window define kar pao, state ke add/remove operations likh pao, valid condition derive kar pao, aur complete Java code ko dry run karke justify kar pao.

End of handbook • Practice every implementation in your own Java IDE.